FinReason — Technical Blueprint

Deliverable 1.2: From Pitch to Prototype

Om Sanjaykumar Patel — MS in AI Systems, University of Florida — Spring 2026

a. Problem Context and Project Summary

Financial analysts spend hours extracting and computing figures from earnings reports—year-over-year revenue growth, operating margin changes, percentage-of-total breakdowns. Large language models promise to automate this, but they consistently fail at the math: they hallucinate numbers, confuse table rows, and skip arithmetic steps. FinReason addresses this by applying reinforcement learning with verifiable rewards (GRPO) to a small LLM, training it to produce correct numerical answers over real S&P 500 financial filings. The reward is simple and deterministic: right number = +1, wrong = 0. No learned reward model, no human annotation—just ground-truth answer verification.

b. Dataset

Property	Details
Source	**FinQA** (Chen et al., EMNLP 2021). HuggingFace: ibm-research/finqa
Type	Tabular + textual financial data with numerical QA pairs
Size	8,281 QA pairs over 2,789 S&P 500 earnings reports
Format	JSON with fields: pre_text, table (2D array), post text, qa (nested dict with question, exe ans, program)
Access	Public, Creative Commons 4.0. No institutional agreement. Downloads via datasets library in seconds.
Split	Train: ∼6,200 / Validation: ∼880 / Test: ∼1,150
Secondary	**TAT-QA** (Zhu et al., ACL 2021): 16,552 questions for out-of-distribution evaluation

Preprocessing Challenges

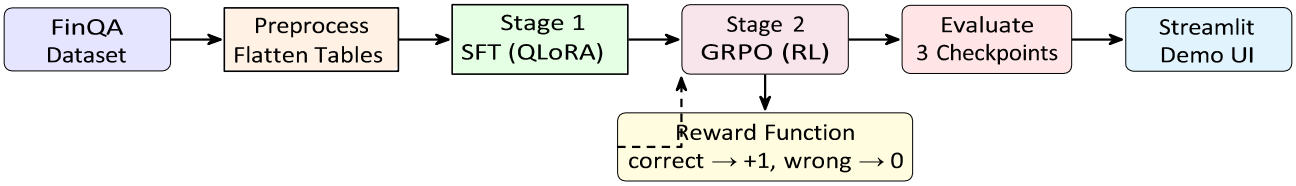
Tables must be flattened from 2D arrays into pipe-delimited text strings. Contexts vary from 200 to 4,000+ characters and must be truncated to fit 1,024 tokens. Answers appear in diverse formats ($1.45B, 1,452 million, (3.5) for negatives) requiring a robust normalization pipeline.

Ethical / Privacy Considerations

All data is derived from publicly filed SEC documents. No PII, no proprietary data, no scraping. The dataset was created by financial experts at UPenn and published through peer review.

c. Planned Architecture

Pipeline Diagram



Algorithms and Frameworks

Base Model: Qwen2.5-1.5B-Instruct in 4-bit NF4 quantization (fits 8 GB VRAM). Fallback: Phi-4-mini (3.8B) on Colab.

Stage 1 (SFT): QLoRA fine-tuning (rank 16, α=32) using HuggingFace TRL's SFTTrainer. Teaches the model to parse financial tables and produce short answers.

Stage 2 (GRPO): Group Relative Policy Optimization via TRL's GRPOTrainer or Unsloth. For each question, 4 completions are sampled, scored by the deterministic reward function, and policy-updated using z-score normalized advantages. The prompt instructs step-by-step reasoning in <think> tags.

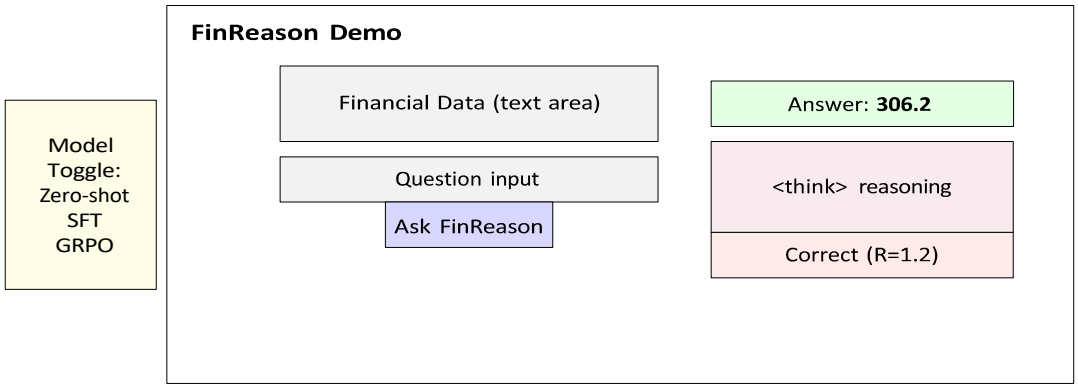
Framework Stack: PyTorch 2.x, HuggingFace Transformers, TRL, PEFT, bitsandbytes, Unsloth (optional), Streamlit.

d. User Interface Plan

Input/Output Design

Component	Description
User Input	Paste or upload financial table/text + type a natural language question
Model Output	Extracted numerical answer, displayed prominently
Reasoning Trace	If the model produced <think> tags, the step-by-step reasoning is shown in an expandable info box
Ground Truth Check	Optional: user enters the correct answer, system shows / and the reward score
Checkpoint Toggle	Sidebar radio buttons to switch between Zero-Shot / SFT / GRPO and compare outputs live
Results Dashboard	Bottom section shows accuracy metrics and all 4 analysis figures from the evaluation

Wireframe



Technology: Streamlit. Already implemented in ui/app.py. Runs with streamlit run ui/app.py.

e. Innovation and Anticipated Challenges

What's new: GRPO has been validated on clean math benchmarks (GSM8K, AIME). Applying it to real-world financial tables—with noisy formatting, unit ambiguity, and domain-specific computation—tests whether RLVR generalises beyond synthetic problems. Running it on a 1.5B model tests the lower bound of where RL-based reasoning works.

C1. Reward sparsity. If the 1.5B model rarely produces correct answers, GRPO gets zero gradient signal. *Mitigation:* SFT warmup first; curriculum ordering (easy→hard questions); increase group size from 4 to 8.

C2. Fragile number parsing. Financial outputs like "$1.45B" or "(3.5)" must be parsed correctly or the reward function gives corrupted signal. *Mitigation:* Exhaustive unit testing on 200+ edge cases before training.

C3. TRL API instability. The GRPOTrainer API has changed between TRL versions. *Mitigation:* Pin TRL version in requirements.txt; maintain a manual GRPO fallback loop.

f. Implementation Timeline

Week	Focus	Expected Outcome
Apr 1–5	Data loading, exploration, environment setup, reward function testing	Working setup.ipynb with verified data pipeline, all self-tests passing
Apr 5–12	SFT training (Stage 1), zero-shot baseline, SFT evaluation	Checkpoints 1 and 2; measurable accuracy improvement over zero-shot
Apr 12–19	GRPO training (Stage 2), reward tuning, hyperparameter search	Checkpoint 3; GRPO accuracy measured; training curves plotted
Apr 19–26	Streamlit UI integration, ablation analysis, TAT-QA OOD test, figures	Interactive demo running; all 4 report figures generated
Apr 26–29	Final report writing, code cleanup, presentation prep	Completed project, polished repo, presentation ready

g. Responsible AI Reflection

Fairness: FinQA is derived exclusively from S&P 500 companies—large-cap US firms. The model may not generalise to financial statements from smaller companies, emerging markets, or non-English filings. Any deployment should be validated on the target population.

Transparency: The <think> reasoning traces provide a degree of interpretability: users can see *why* the model produced a particular number, not just the number itself. The Streamlit UI exposes this reasoning directly.

Environmental cost: Total training compute is modest: ~6–8 GPU-hours on a single consumer GPU, well below the threshold of frontier model training. The use of 4-bit quantization and QLoRA minimises energy consumption relative to full fine-tuning.

Misuse risk: A model that answers financial questions could produce incorrect outputs that are acted upon in real financial decisions. The system should be positioned as an *assistant* tool, not an autonomous decision-maker, and all outputs should carry a disclaimer that they are AI-generated and require human verification.

Github Link: https://github.com/OmSPatel20/FinReason.git